An Efficient Consensus Protocol for Real-Time Permissioned Blockchains under non-Byzantine Conditions*

Gengrui Zhang[1,2] and Chengzhong Xu[1,3]

[1] Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences
[2] Shenzhen College of Advanced Technology, University of Chinese Academy of Sciences
[3] Department of Electrical and Computer Engineering, Wayne State University, USA
{gr.zhang, cz.xu}@siat.ac.cn

Abstract. Blockchains are increasingly used in the collaboration between business as a trusted distributed ledger. Coping with massive data transactions raises the requirement of real-time safety of blockchains. The celebrated Raft protocol has limitations of being a consensus protocol for permissioned blockchains where a strong consistency is needed between clients and servers. In this work, we propose a new consensus protocol called Dynasty which ensures the real-time safety and the liveness under all non-Byzantine conditions. We design and implement a three-layer permissioned blockchain framework which tolerates f failures with $2f+1$ correct servers based on Dynasty. We demonstrate the blockchain as a service in an application of used-vehicle trading management and evaluate the performance of the blockchain framework in terms of throughput and latency. Experimental results show that while the latency in different scales of the system increases as expected, the number of committed transactions per second stabilizes at a point within less than 8% difference after a warming-up period.

1 Introduction

Since Bitcoin [18] has been widely used to exchange for currencies, products and services, the blockchain technology where the transactions are stored in a distributed ledger accomplishes the solution to validate transactions without the presence of any trusted authority. At present, the applications based on blockchains have been extended from digital currency to other financial applications such as Ripple [20], Dash [8], BlackCoin [13] and Hyperledger Fabric [5]. Using blockchains makes it unnecessary to use a third party to provide credit support for the transactions. Each node that joins the system maintains the same chain at any verifiable time and adds the same block when the system reaches a consensus. Depending on the applicable scenarios, the blockchains are divided into two categories, namely, permissionless and permissioned.

* Published as a conference paper at GPC 2018

Acting as a real-time trusted system is becoming an urgent requirement of blockchains. The real-time safety requests that the blockchain systems are able to provide the real-time consistency of every committed transaction at any time since it has committed. The permissionless blockchains use Proof-of-X [2,13,18] based protocols to provide consensus services with no management of the node identity. For example, the Proof-of-Work protocol, firstly used in Bitcoin, guarantees the correctness when no party can control over 51% of the computing power and offers a good scalability of nodes and clients but a poor performance in throughput and latency on a widely-open ledger. Due to the high latency and the low throughput, permissionless blockchains can not meet the requirement in building a real-time service blockchain driven by a high volume of transactions. In addition, the huge energy consumption of Proof-of-Work based protocols in solving cryptographic puzzles reduces the value of the commercial use. On the contrary, the IDs of all other servers are known in the permissioned blockchains since the nodes added in the system are already licensed by the node management. The consensus protocols such as Paxos [15], PBFT [6] and Raft [19] manage the fault-tolerance based on the theory of the state-machine replication, which makes a greatly improved performance in throughput and latency but brings a limited scalability [21]. The above protocols make the permissioned blockchains are well suited to commercial applications such as cross-border transactions between banks and smart contrast between business cooperation. Raft is used by R3 Corda [4], QuorumChain, Hyperledger Kafka [5] to provide a fast blockchain consensus service with high throughput and low latency of transactions. Raft guarantees the tolerance in any $f < n/2$ crashes on n nodes in total.

However, the Raft protocol can not ensure the real-time safety in terms of committing transactions. A majority of cluster does not commit the transaction when the client receives a notify from a Raft-based blockchain system. Although Raft can guarantee the ultimate consistency of the commitment, the real-time consistency also need to be considered in building permissioned blockchains especially when the transactions have sequential dependencies.

In this paper, we present an efficient consensus protocol called Dynasty which ensures the real-time safety and the liveness in building permissioned blockchains. Furthermore, Dynasty performances a good throughput and scalability with $O(N)$ messages in each round. We design and implement a three-layer permissioned blockchain framework to serve real-time requests of clients based on Dynasty protocol. The blockchain could tolerate f failures with $2f + 1$ correct servers under all non-Byzantine conditions such as crash, network delays, packet omission and record tampering. We believe that the trading business model based on this blockchain framework is superior to those centralized, both in data protection and transaction transparency.

In summary, we made the following contributions:

1. *Dynasty consensus protocol.* The new protocol is designed for building real-time permissioned blockchains. Its properties guarantee the real-time safety

and the liveness under all non-Byzantine conditions. It handles the consensus with a message complexity of $O(N)$ on broadcasts.

2. *Application in used-vehicle trading.* The implemented trading model for managing the trades of used vehicles is a distributed ledger based on the blockchain framework. It makes the trading transactions recorded chronologically and irreversible. The evaluation suggests that the blockchain based on Dynasty is more practicable in terms of throughput and latency.

The remainder of this paper discusses the related work of consensus problems and blockchain applications (Section 2), describes the design and analysis of the Dynasty consensus protocol (Section 3), demonstrates the blockchain framework and its three layers (Section 4), and presents the case study with a used-vehicle trading application and the evaluation (Section 5).

2 Related Work

2.1 Consensus Protocols in Blockchains

The blockchains, whether permissionless or permissioned, are distributed ledgers that rely on consensus protocols to achieve agreements. The consensus problem is to get a group of n processes in a distributed system to agree on a value. A consensus protocol is an algorithm that produces such an agreement. Correct consensus protocols must satisfy the following three conditions [1]: agreement, termination and validity, which means that all processes that decide choose the same value, all non-faulty processes eventually decide, and the common output value is an input value of some process.

Consensus Protocols for Permissionless Blockchains. Bitcoin [18] uses the Proof-of-Work protocol to avoid charging real money by requiring senders to demonstrate that they have expended processing time in solving a cryptographic puzzle. The work-based certification protocols require a lot of consumption to record transactions so that fraudsters pay much more than the reward when they modify the history record. In permissonless blockchains, there are also some other protocols such as Bitcoin-NG [10], Proof-of-Stake [13] and Proof-of-Activity [2] accomplish the consensus even more efficient and applicable in some additional requirements than the PoW protocol.

Consensus Protocols for Permissioned Blockchains. Unlike the permissionless blockchain that opens to the whole network, the nodes that join the system are not required a permission, permissioned blockchain is more suitable for specific cooperative transactions as it faces the users with a certain permission. Some consensus protocols such as Paxos [7, 15, 16], PBFT [6], XFT [17] and Raft [19] could handle the consensus between servers in permissioned blockchains. Although the celebrated PBFT protocol achieves the Byzantine agreement [6, 11, 14], it brings a lot of problems such as the low efficiency of

message delivery, the heavy capacity of network and the additional costs of the strong requirement of Byzantine broadcast. PBFT tolerates f Byzantine faults with the cost of $O(N^2)$ by broadcasting signed messages. However, the Byzantine problem rarely happens in such a system where the nodes are licensed to join in. When using PBFT as the consensus protocol of permissioned blockchains, the additional costs in terms of resources and message delivery cause a great loss on the throughput and scalability.

Some protocols such as Paxos and Raft have a better efficiency than PBFT under non-Byzantine conditions. Paxos ensures that a single value among the proposed values is chosen [3, 15]. Although the Basic Paxos meets the requirements of safety and liveness by its two phases of operating actions in accepting proposed values, the livelock could happen when a next propose phase always comes before the previous accept phase, and such disadvantages of Basic Paxos make it not practical enough to build a real-time service blockchain.

Raft guarantees the election safety, leader append-only, log matching, leader completeness and state machine safety. Those proofs are given in its paper in Section 5 [19]. The features of log replication and leader election make it superior to other consensus algorithms, both for understanding and as a foundation for implementation of blockchains.

Although the Raft protocol has so many advantages of being a consensus protocol, it does not meet the requirements of the real-time safety of permissioned blockchains driven by transactions. We will discuss the limitations of Raft in building real-time blockchains in the next Section.

2.2 Blockchain Applications

Blockchain technology benefits shared economy applications as a financial technology (FinTech) in providing a wide variety of services spanning areas such as payment, financial management, insurance business and asset registration. [12] points out that contracts, transactions, and the records of them are among the defining structures in our economic, legal, and political systems. Blockchain adoption has the power to transition new and existing models of insurance, including P2P insurance, parametric insurance and microinsurance, into a new digital age.

The blockchain-based smart contracts have been rapidly developed in the financial economy. These contracts can be partially or fully executed or enforced without human interactions. A smart contract could be enabled by scalable program-instruction that initializes and executes an agreement. These automate payments and the transfer of currency or other assets as negotiated conditions are met. Firms could be built on contracts, from incorporation to buyer-supplier relationships to employee relations [12]. In Hyperledger fabric [5] and Ethereum [22], smart contracts are mostly used more specifically in the sense of general purpose computation that takes place on a blockchain.

3 Design and Analysis of Dynasty Consensus Protocol

In this section, we present the Dynasty protocol in detail. First, we discuss the limitations of Raft in being a consensus protocol for permissioned blockchains. Then we demonstrate the Dynasty protocol and its features in terms of real-time safety, liveness and fault-tolerance.

3.1 Raft and Its Limitations.

Raft protocol manages the agreement of producing the same outputs from the identical sequences of commands of the replicated state machine. In Raft, each server runs one state of leader, follower, or candidate at any given time. In normal operations there is exactly one leader and all of the other servers are followers. The system time of Raft is divided into terms. Terms are numbered with consecutive integers, and a single leader manages the cluster until the end of the term.

Leader Election. The servers begin with the initial state as a follower. From Fig. 1-(a) we can see that when a timeout happens on a specific follower, it becomes a candidate and starts an election by asking other servers to vote in its term. If the candidate is elected by a majority, it must has received more than half servers' tickets, then it becomes the new leader of the current term and begins to take the authority of leading the consensus progress with other servers.

Log Replication. Raft uses remote procedure calls (RPCs) to communicate between servers. During log replications, the leader appends commands to its log as a new entry, then issues AppendEntries RPCs in parallel to each of the other servers to replicate the entry. The arguments in AppendEntries RPC are: *term, leaderId, prevLogIndex, prevLogTerm, entries[], leaderCommit.* The *term* is the leader's current term; the *leaderId* is used for redirecting clients to a new leader when the crash happens on the elder leader; the *prevLogIndex* is the index of log entry immediately preceding new ones; the *prevLogTerm* is the term of *PrevLogIndex* entry; the *entries[]* is log entries to store commands, and it could be empty for heartbeat; the *leaderCommit* is the leader's local commit index. Formally, we denote S_l^k with $\{term, L_{id}, Index_{log}^{prev}, term_{log}^{prev}, Index_{cmt}^{l}\}$ to represent the arguements in AppendEntries RPC in a round k. Thus $S_l^k(\Omega)$ stands for the *entries[]* in AppendEntries RPC is a record Ω. The $S_l^k(\perp)$ represents the heartbeat that contains an empty *entries[]*.

Fig. 1-(b) shows the normal operations in a 4-node Raft cluster in round k and $k+1$. In round k, Ω is proposed to the leader by a client. Then, the leader broadcasts $S_l^k(\Omega)$ to followers: f_1, f_2, f_3. Followers log Ω and send back $R_f^k(\Omega)$ to leader. When the leader receives a majority replies of $R_f^k(\Omega)$, the consensus is reached for committing Ω. By this time, leader increases $Index_{cmt}^{l}$

and notifies the proposed client that Ω has been committed. In Raft, if a majority of the cluster has responded the return value, the commit is made, otherwise the commit is refused.

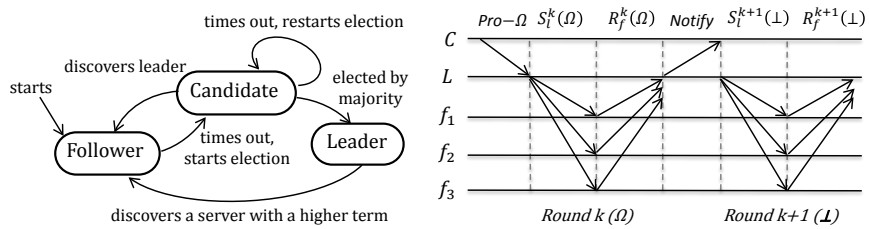

Fig. 1: The three states of server (a) and normal case operations of log replication (b) of Raft

The Safety Limitations. In a Raft-based blockchain, there is no majority server has committed Ω before the proposed clients are notified by leader. Since followers are only passive to receive the message and give the return value, the followers always increase their $Index^f_{cmt}$ by synchronizing the $Index^l_{cmt}$ in the next round. As a result, Ω cannot be committed on leader and followers in the same round.

In Raft, the *unsafety condition* happens when the proposed client is notified in round k until a majority cluster commits in round $k+1$. In log replication, followers never know the consensus result in round k, which results in that the $Index^f_{cmt}$ won't be increased until followers synchronize the $Index^l_{cmt}$ in $S_l^{k+1}(*)$ (where * represents either Ω or $\perp$) in round $k+1$ even the leader has notified the proposed client. What's worse, the unsafety time could be much longer in the case that the leader l crashes after it notified the proposed client. During a period between l has crashed and a new leader l^* with a higher term t^* ($t^* > t$ in l) is elected, the unsafety time gets much longer since there is no $S_l^{k+1}(*)$ for followers to synchronize their $Index^f_{cmt}$ from $Index^l_{cmt}$.

Although the followers could eventually commit Ω according to the properties of Raft, the *unsafety condition* still cannot be tolerated when we use the real-time blockchains as a service. For example, for smart contracts, we assume two contracts A and B where A could be paid individually but B depends on A has been paid first. In above conditions, when the notice that A was accomplished is sent to the proposed client, the contract B could be proposed to the system. At this time, actually, there is no cluster has committed A besides the current leader but B is proposed by client. If the current leader crashes, until a new leader l^* is elected, the log replication stops but the transactions are blocking in the committing queue. We do not allow those transactions that have dependency relationship block in the same committing queue. Since the new leader

issues $S_{l^*}(*)$ in parallel to each server to replicate entries, the contract B could be accomplished before A on a majority servers. As a result, the dependence relationship of $A \rightarrow B$ changes to $B \rightarrow A$.

We need a confirmation that a majority of the cluster has stored the record and provides an interface for querying requests before the leader notifies external clients. The Dynasty protocol perfectly solves the *unsafety condition* when we design real-time service blockchains.

3.2 Dynasty Consensus Protocol

Dynasty is a consensus protocol which ensures the real-time safety and the liveness. When designing Dynasty, we refer to the basic idea of strong leader and leader election features of Raft. The Dynasty protocol is totally different in the process of log replication from Raft, we present a new method called two-step commit to ensure the safety property to meet the requirements of building a real-time permissioned blockchain.

Overview. The Dynasty protocol uses a log phase and a commit phase to replicate a proposed record. In the log phase, we denote $S^k_{l\ log}$ with $\{Sym_{log}$, $term$, L_{id}, $Index^{prev}_{log}$, $term^{prev}_{log}$, $Index^{l}_{cmt}\}$ where Sym_{log} is a symbol character to distinguish the RPC type. When a client proposes a record Ω to leader, the leader broadcasts $S^k_{l\ log}(\Omega)$ to other servers to start a consensus process. In the commit phase, the symbol character in $S^k_{l\ cmt}(\Omega)$ changes to Sym_{cmt}. We also use $S^k_l(\perp)$ where the symbol is $Sym_{\perp}$ to stand for the heartbeat in round k to maintain the authority of the leader.

In Dynasty, heartbeat is only used for resetting followers' timer. Followers never synchronize the $Index^{l}_{cmt}$ through $S^k_l(\perp)$ and the arguments of $S^k_l(\perp)$ change to $\{Sym_{\perp}$, $term$, L_{id}, $Index^{prev}_{log}$, $term^{prev}_{log}\}$. The $S^k_l(\perp)$ is sent between a fixed interval $Time_{\perp}$ from the leader. To maintain the liveness and stability of the system, we set $Timeout_f \gg Time_{\perp}$ to tolerate some network delays and guarantee the timeout never happens on followers before the $S^k_l(\perp)$ comes in normal cases.

The *Legality Check (LC)* for receivers in round k includes message-type validity and leader state validity. The follower resets its timer when $S^k_{l\ log}(\Omega)$, $S^k_{l\ cmt}(\Omega)$ or $S^k_l(\perp)$ passes the LC. The requirements are as following:

- $term$ of receiver $\leq term$ in S^k_l. (both for $log, cmt, \perp$)
- $\exists term$ of Ω at $Index^{prev}_{log}$ matches $term^{prev}_{log}$.

Two-step Commit. Dynasty uses two-step commit to ensure that leader always commits a record after a majority cluster has done. In Fig. 2, we illustrate a process of committing Ω in round k. The two-step includes two phases: *Log Phase* and *Commit Phase*, following shows the details.

First Step

- A client (C) sends a propose request with record Ω to leader (L).
- *Log Phase:*
  - L broadcasts $S^k_{l\ log}(\Omega)$ to other servers.
  - If the received $S^k_{l\ log}(\Omega)$ passes the LC, followers send back $R^k_{f\ log}(\Omega)$ to L indicates that Ω has logged and agrees to commit.

Second Step

- *Commit Phase:*
  - L broadcasts commit requests $S^k_{l\ cmt}(\Omega)$ to followers when the number of received $R^k_{f\ log}(\Omega)$ is more than half.
  - If the received $S^k_{l\ cmt}(\Omega)$ passes the LC, the follower: commits Ω ; increases $Index^f_{cmt}$; sends back $R^k_{f\ cmt}(\Omega)$ to L.
  - Once the number of received $R^k_{f\ cmt}(\Omega)$ more than $n/2$, the leader L: commits Ω ; increases $Index^l_{cmt}$.
- The leader L notifies the proposed client that Ω has been safely committed.

During the normal case, a committed record involves $4N$ messages in total as in each step, the leader only broadcasts $O(N)$ messages and the followers only passively sends back to the leader with $O(1)$ message. Thus, the Dynasty protocol reduces the message complexity from exponential level to a constant degree than PBFT.

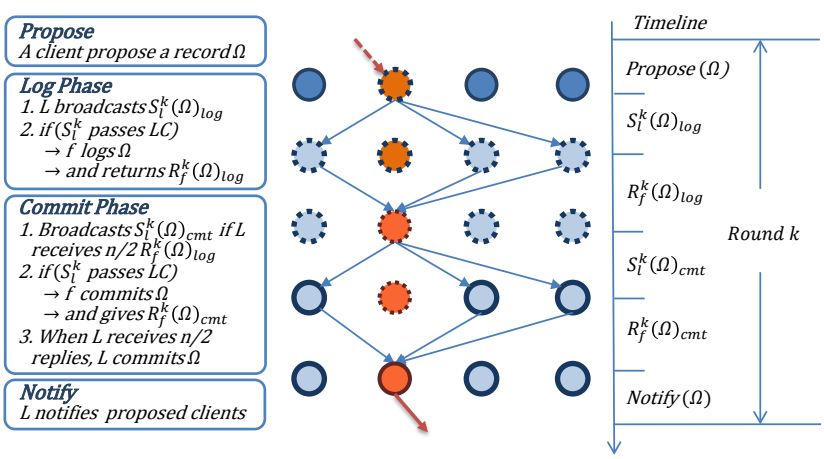

Fig. 2: Two-step commit ensures that the leader always commits after a majority cluster has committed. Timeline shows the happens-before process.

View-change. The current leader in the blockchain may crash down or have a high network latency, which may result in a time that the blockchain system could neither have any communication to external clients nor proceed any consensus process since the followers only passively response to the leader. The view-change begins on these conditions, a new leader needs to be elected to ensures the liveness of the system so as to the blockchain never goes to a termination. Referring to the idea of the leader election in Raft, Dynasty protocol also defines three states of each server: leader, follower, or candidate. A follower increases its term t to $t+1$, starts an election and becomes to a candidate when its timer has a timeout. The new leader comes from the candidate which has received a majority votes in the whole cluster must have the highest term value. We do not modify the election part (Section 5.2 in [19]) of Raft since it does not have a bad influence in building the real-time blockchain in this work. The blockchain based on Dynasty cannot provide any service during the election time.

3.3 Analysis of Dynasty Protocol

The Dynasty protocol ensures the safety and the liveness properties in building real-time service permissioned blockchains. The two-step commit provides a real-time safety promise in building blockchains than Raft. This helps the Dynasty protocol reduces the conflicts between the interactions with clients and makes the transaction driven more practicable.

Property 1. (Safety) Two-step commit ensures that a majority of cluster has committed before the leader sends a notice to the proposed client.

Proof. In Dynasty, the *unsafety condition* never happens. In normal case, as shown in Fig. 2, the timeline shows that the leader commits a record Ω when it has received majority $R^k_{f_cmt}(\Omega)$ replies, so that the notify won't be sent before two-step commit has been accomplished. In the case of leader crashes, the two-step commit also ensures the real-time safety. In the log phase, if the leader crashes, the consensus fails and a new leader l^* continues to broadcast the $S^{k+1}_{l^*}(\perp)$ after being elected, the system aborts the second step of committing Ω. There is no follower commits the logged Ω and the safety is ensured. In the commit phase, if the leader crashes and a new leader must be elected according to the election method. The elected leader must the one that has committed Ω since in the *Legality Check*, it guarantees the leader's $Index^{prev}_{log}$ is more up-to-date than a majority server in the system. The new leader l^* broadcasts ${S^{k+1}_{l^*}}_{log}(\Omega)$ to form a new process of committing Ω. Above all, the leader always commits after a majority cluster has committed despite of any conditions. □

The liveness is promised by non-stop elections until a new leader is elected in the view-change. A new election must start when a follower's or a candidate's timer triggers a timeout. Assume that the current leader l crashes, after at most $min\{Timeout_{f_i}\}$ time passes, a candidate begins to start an election

with a higher term $t^*(t^* > t)$. When multiple candidates get the same number of votes, the election in term t^* fails but a new election must be start after the $min\{Timeout_{Candi_i}\}$. Since the term of the system never decreases on any conditions, the system never goes back to a previous time. In order to avoid a long period of election, each server's timer is initialized by random times.

When we use Dynasty to implement a real-time permissioned blockchain, after the consensus process, the following works step to store the record from memory to disk and provide an interface for clients to handle query quests.

4 Design of Blockchains Based On Dynasty Protocol

In this section, we demonstrate the structure of the blockchain in detail. In the fist subsection, we present the basic idea of the three-layer design, the detailed block structure and the query method. In the second subsection, we give the fault-tolerance discussions of the blockchain.

4.1 Three-layer Design

The designed blockchain framework is divided into two parts: cluster and clients. The servers with states of leader and follower make up the cluster where the records could be stored after being committed. The client communicates with the internal system by sending propose-request and query-request to the current leader.

The three-layer design is shown in Fig. 3, in the consensus layer, the leader is not only a bridge that communicates with the clients in terms of receiving requests and sending notices, but also manages the consensus with other servers in the blockchain. The storage layer packs the committed records to blocks and stores them on local server once the consensus has been reached. The query layer responds to the query requests from the clients and gives back the result according to the $QMethod$.

Consensus Layer. The protocol in consensus layer is the designed Dynasty in Section 3. Consensus layer handles the proposals from clients and maintains the consistency between servers in the blockchain.

Storage Layer. The principle of the block structure in the storage layer is packing a committed record into single block concatenated with a unique *BlockId*. We use the hash of the block's address to act as the unique Id. Each new block holds its own hash and pervious block's hash, and links to a previous one by the previous block's hash up to the first. Fig. 3 shows a chain that owns three blocks with one being built. The block 3 is waiting to pack the next record from the consensus layer. Block 2 links to Block 1 and these two are both available to the query layer. In this way, a unique chain is formed on different servers in the whole system.

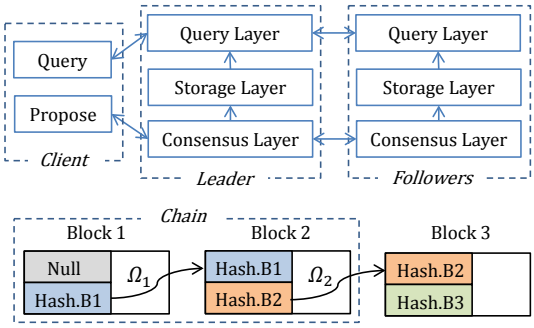

Fig. 3: The three-layer design. The storage layer saves the value passed from the consensus layer. The query layer provides an interface between clients and servers.

Once a record is committed, it has been stored in a distributed ledger on different servers. This single record block structure makes it quicker to add a new record into a chain, and the *BlockId* makes it more efficient to the query service to locate a certain block.

Query Layer. In the query layer, the blockchain believes that the correct result is the most consistent result. When the current leader notifies the proposed client, according to property 1, majority servers must have stored the record and the blocks are accessible for query layer to obtain. We use the $QMethod$ strategy that chooses a value among a group of different values according to the majority principle to return the final result. Thus, the $QMethod$ gives the correct result when the correct nodes are not less than $f + 1$ when f records are faulty.

4.2 Fault Tolerance.

With above designs, this blockchain can tolerant f faults when the correct nodes are not less than $2f + 1$. For example, in a 5-node blockchain of Fig. 4, $S0$ is the leader and the other 4 are followers. $S3$ crashes and $S1$ gets a storage fault. In each step of the consensus process, the leader still gets enough correct replies ($R^k_{f\,log}$ and $R^k_{f\,cmt}$) from other servers. Thus, the consensus still could be reached. In terms of querying process, the $QMethod$ also gives the correct result since correct servers are more than faulty ones.

In our Dynasty based blockchain, if f faults all happens on the consensus stage, the blockchain needs $f + 1$ well-behaved servers to ensure the enough replies. Similarly, the $QMethod$ also need $f + 1$ correct backups to guarantee the correctness of final result when there are f storage-faulty servers in the blockchain. The worst case is that those storage faulty nodes are non-crash servers so that the safety must be guaranteed at least $2f + 1$ correct servers.

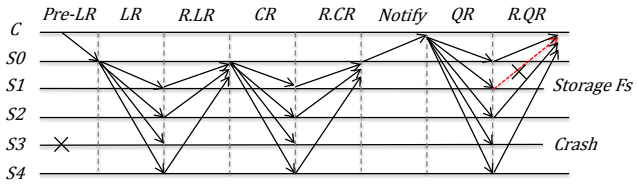

Fig. 4: A five-node blockchain based on Dynasty. Server $S3$ crashes, server $S1$ has a storage fault.

5 Case Study

We have implemented the blockchain with roughly 2000 lines of java code, not including tests, comments, or blank lines. We use multi-thread to send and receive packets, distinguish message types, countdown fixed time and other functions. The source code is freely available on GitHub [23]. The reminder of this section we present the application of used-vehicle trading model based on our blockchain framework and the evaluation of throughput and latency.

5.1 Used-Vehicle Trading Model

The traditional business management in used-vehicle trading is centralized so that transactions between buyers and sellers must get through a third party (agency). The buyers can only obtain vehicles' information through those agencies, which may breed business fraud cases such as the concealment of the vehicle's damage, the fake years of used and the missing information of transfers.

In a blockchain based trading management, the transactions and the vehicle's insurance information are recorded on a open ledger. No one can change the transaction records in the blockchain unless he can control more than half the nodes in the system. Each participating node proposes the information of vehicles, each record is stored on a majority of cluster after the consensus approach. Sellers and buyers are able to read the records by proposing query-request, and we do not need a third party to provide proofs of vehicles' information anymore.

5.2 Evaluation and Analysis

We measured the throughput and the latency of the designed blockchain on a cluster of 4, 8, 12 and 16 nodes. Each server has an E5-2630 2.40GHz CPU, 64 GB RAM, 2 TB hard drive, running on Ubuntu 14.04.1, and connected to the other servers via 1GB switch. In out tests, we do not consider any case of node failure, but allow the loss of packet. All the results are averaged over 5 independent runs.

Throughput. We measured the throughput as the number of successful transactions per second (TPS). The client sends continuous requests in order to saturate the network capacity to achieve a maximum throughput. We noticed that a

warm-up period always appears at the beginning of our tests. During the warm-up period, since the transactions are in the order requested by the leader in turn sent to the servers in the cluster, the network capacity does not reach the peak at the beginning of the test. After the network has been saturated to the maximum, the leader and other servers maintain a smooth throughput that stabilizes at a point within tiny fluctuations.

From Fig. 5-(a) we can see that along with the increasing scale of the cluster's size, either TPS with or without the warm-up period shows a downward tendency due to the requirement of more replies to reach the consensus. In our 240 second tests, the 4-node blockchain commits and stores averaged 1.8×10^5 transactions in total and the TPS averaged at 764 which is the maximum in all test cases. When the cluster's size increases to 16, the number of committed transactions falls to 1.3×10^5 and the TPS decreases to an average of 543.

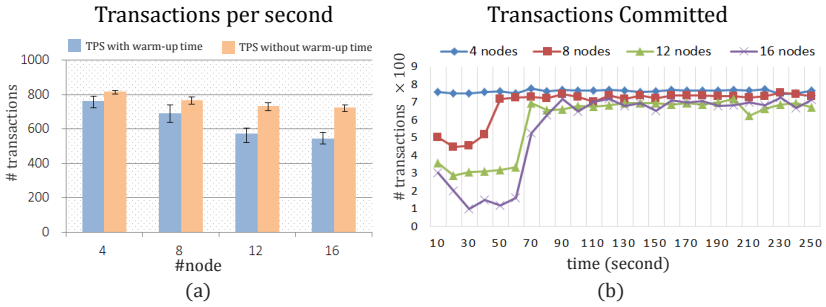

Fig. 5: The throughput due to the blockchain based on Dynasty.

We have analyzed the message complexity of Dynasty protocol in Section 3. During normal cases, a committed record involves $4N$ messages in the consensus layer as the two-step commit method takes $O(N)$ message in each step. The constant degree of the message complexity could be verified from Fig. 5-(b), the average TPS has been measured in a interval of 10s, and the throughput reduces less than 8% in each increasing 4 nodes of the cluster's size after the warm-up period. The peak performance is measured as the highest TPS per 40s as shown in Fig. 6-(a) that the increased cluster size is the major factor that causes the degradation during the warm-up period (first 120s). After that, the cluster size alleviates the influence and the performance gap between different sizes of clusters gets smaller.

Latency. We measured the latency as the response time of each successful transactions. According to Dynasty protocol, each transaction needs two round $O(N)$ broadcasts, which makes the latency affected more by network delay. Fig.6-(b) shows that the latency gets marked higher as expected along with the increasing sizes of the scale.

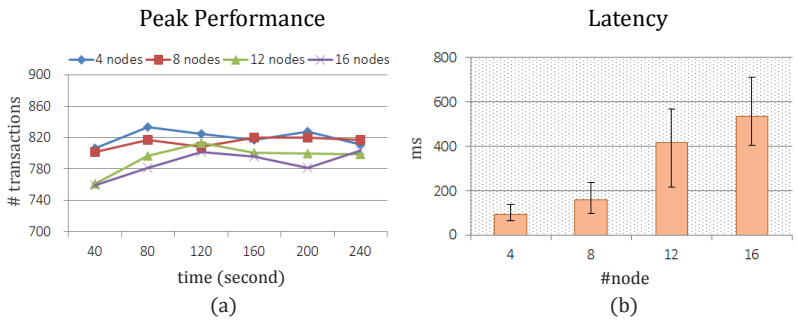

Fig. 6: The peak performance and the latency of the blockchain based on Dynasty. (a)The peak performance. (b)The latency.

The above evaluations show that while the latency in different scales of the system increases as expected, the number of committed transactions per second stabilizes at a point within less than 8% difference after a warming-up period. The throughput and the latency are both severely affected by the cluster's size, which indicates that measuring a balance between throughput and scalability is an urgent problem in blockchain-based applications. The work in [9] measured the scalability of Hyperledger and the test stopped working beyond 16 servers. Although in our work we used Dynasty protocol which reduces the broadcast cost to build the blockchain framework, we still observed that the performance reduces when the cluster size becomes larger.

6 Conclusion

Blockchains benefit commercial applications as a distributed ledger where transactions could be committed without the presence of any trusted authority. The permissioned blockchain works in reaching consensus, replicating state, broadcasting requests and storing transactions on server.

In this paper, we discussed the limitations of the real-time safety of Raft in terms of being a consensus protocol in blockchains. We designed and analyzed a new consensus protocol called Dynasty with its two-step commit method to ensure the real-time safety. We demonstrated a three-layer blockchain framework based on Dynasty, which requires $2f+1$ correct servers to tolerate f failures under non-Byzantine conditions. We implemented the framework in a used-vehicle trading application and measured the evaluations of the blockchain on several clusters with different size. We observed that in our approach, the size of the blockchain for each 4-node increases, the throughput only takes 8 percentage reduction even the latency nearly doubly increases.

7 Acknowledgment

The authors would like to thank the anonymous reviewers for their insightful comments and valuable suggestions. This work was supported by the China National Basic Research Program (973 Program, No. 2015CB352400), NSFC under grant U1401258, Science and Technology Planning Project of Guangdong Province (2015B010129011, 2015A030310326), the Basic Research Program of Shenzhen (JCYJ20150630114942313).

References

1. Aspnes, J.: Randomized protocols for asynchronous consensus. Distributed Computing 16(2), 165–175 (2003)
2. Bentov, I., Lee, C., Mizrahi, A., Rosenfeld, M.: Proof of activity: Extending bitcoin's proof of work via proof of stake [extended abstract] y. ACM SIGMETRICS Performance Evaluation Review 42(3), 34–37 (2014)
3. Bolosky, W.J., Bradshaw, D., Haagens, R.B., Kusters, N.P., Li, P.: Paxos replicated state machines as the basis of a high-performance data store. In: Symposium on Networked Systems Design and Implementation (NSDI). pp. 141–154 (2011)
4. Brown, R.G.: Introducing r3 corda: A distributed ledger designed for finanial services, 2016 (2017)
5. Cachin, C.: Architecture of the hyperledger blockchain fabric. In: Workshop on Distributed Cryptocurrencies and Consensus Ledgers (2016)
6. Castro, M., Liskov, B., et al.: Practical byzantine fault tolerance. In: OSDI. vol. 99, pp. 173–186 (1999)
7. Chandra, T.D., Griesemer, R., Redstone, J.: Paxos made live: an engineering perspective. In: Proceedings of the twenty-sixth annual ACM symposium on Principles of distributed computing. pp. 398–407. ACM (2007)
8. Dash: https://www.dash.org/
9. Dinh, T.T.A., Wang, J., Chen, G., Liu, R., Ooi, B.C., Tan, K.L.: Blockbench: A framework for analyzing private blockchains. In: Proceedings of the 2017 ACM International Conference on Management of Data. pp. 1085–1100. ACM (2017)
10. Eyal, I., Gencer, A.E., Sirer, E.G., Van Renesse, R.: Bitcoin-ng: A scalable blockchain protocol. In: NSDI. pp. 45–59 (2016)
11. Fischer, M.J., Lynch, N.A., Paterson, M.S.: Impossibility of distributed consensus with one faulty process. Journal of the ACM (JACM) 32(2), 374–382 (1985)
12. Iansiti, M., Lakhani, K.R.: The truth about blockchain. Harvard Business Review 95(1), 118–127 (2017)
13. King, S., Nadal, S.: Ppcoin: Peer-to-peer crypto-currency with proof-of-stake. self-published paper, August 19 (2012)
14. Lamport, L., Shostak, R., Pease, M.: The byzantine generals problem. ACM Transactions on Programming Languages and Systems (TOPLAS) 4(3), 382–401 (1982)
15. Lamport, L., et al.: Paxos made simple. ACM Sigact News 32(4), 18–25 (2001)
16. Lampson, B.: The abcd's of paxos. In: PODC. vol. 1, p. 13 (2001)
17. Liu, S., Viotti, P., Cachin, C., Quéma, V., Vukolic, M.: Xft: Practical fault tolerance beyond crashes. In: OSDI. pp. 485–500 (2016)
18. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system (2008)
19. Ongaro, D., Ousterhout, J.K.: In search of an understandable consensus algorithm. In: USENIX Annual Technical Conference. pp. 305–319 (2014)

20. Schwartz, D., Youngs, N., Britto, A.: The ripple protocol consensus algorithm. Ripple Labs Inc White Paper 5 (2014)
21. Vukolić, M.: The quest for scalable blockchain fabric: Proof-of-work vs. bft replication. In: International Workshop on Open Problems in Network Security. pp. 112–125. Springer (2015)
22. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum Project Yellow Paper 151 (2014)
23. Zhang, G.: Transaction based blockchains. https://github.com/thatisedward/Transaction-Based-Blockchain